LaTeX template `article-template`

Yifei Fan

Contents

1 Introduction **2**

2 Features **2**

2.1 Clean project structure . . . 2

2.2 Convenient figure management . . . 3

2.3 Powerful math macro helpers . . . 4

Math macros declaration 4 • Custom quotient macro 4 • Stacked arrows 5

2.4 Environments and cross-references . . . 5

2.5 Beautiful layout and typography . . . 5

2.6 Git integration . . . 6

3 Usage **7**

3.1 Initialize a new project . . . 7

3.2 Build the project . . . 7

3.3 Modify the metadata . . . 7

3.4 Write the content . . . 8

3.5 Insert a figure . . . 8

3.6 Synchronize template updates . . . 8

4 Some tips **8**

4.1 Filename convention . . . 8

4.2 Cross-references . . . 9

4.3 Citations . . . 9

4.4 Figures . . . 10

References **11**

1 Introduction

Journal of XXCC Unicorn is a journal dedicated to publishing articles written by Xiayu Tan and Yifei Fan. It is published by **XXCC Unicorn Press**, which can be visited at https://xxcc-unicorn-press.github.io. You can also scan the QR code below to access the journal's website directly.

This document is a general-purpose template for writing articles. You can find the source code of this template at [1]. It is compatible with the template in [2], which is a template for turning articles into journal issues.

This template is inspired by [3], which is a beautiful template for writing articles. It is lightweight and easy to use, and it provides a clean and elegant design for your articles.

2 Features

This template provides an out-of-box experience for writing beautiful articles with LaTeX.

2.1 Clean project structure

The project structure is organized as follows:

- `main.tex`: The main entry point of the document.
- `references.bib`: The bibliography file containing all the references used in the article.
- `preamble/`: A directory containing various preamble files for different purposes, such as layout design, reference management, math formatting, etc.
- `content/`: A directory containing the content of the article, organized into different sections and subsections.
- `figures/`: A directory for storing all the figures used in the article.

By organizing the project in this way, it becomes easier to manage and maintain the document, especially as it grows in size and complexity.

It is **recommended** that files are named in lowercase with hyphens as separators. For content files, they can be named according to the section names, such as `fermats-last-theorem.tex` for a section named "Fermat's Last Theorem".

2.2 Convenient figure management

The `\includegraphics` command can now recognize paths relative to `figures/` and `figures/inkscape` directories, without need to specify the full path.

You can put your Inkscape source files in `figures/inkscape` and run `./figures/inkscape/export.sh` to export all the `.svg` files to `.png` files, which can be included in the document directly. If you use `git`, the script will be automatically run before each commit to ensure that the exported `.png` files are always up-to-date with the source `.svg` files.

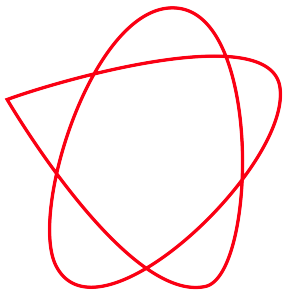

Figure 2.1 / A figure exported from Inkscape.

The figures are automatically centered, so there is no need to write `\centering` every time. It is **recommended** to use `[!ht]` as the figure placement specifier to allow more flexibility in figure placement. An example of including a figure is shown below (see Figure 2.2 for the result):

```
\begin{figure}[!ht]
  \includegraphics[width=0.5\textwidth]{example-image-golden}
  \caption{An example figure.}
  \label{fig:example-figure}
\end{figure}
```

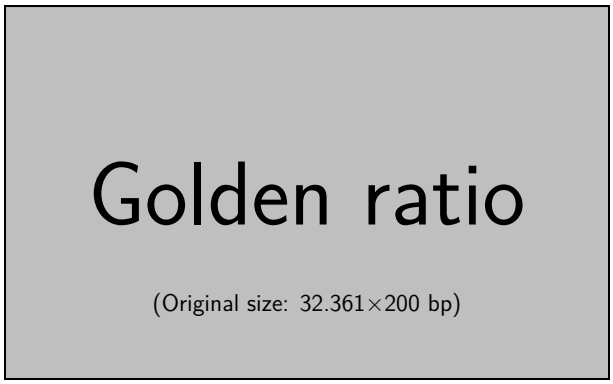

Figure 2.2 / An example figure.

Side captions are supported by the `floatrow` package, and you can use the `\fcapside` command to create figures with side captions. Here is an example of how to use it (see Figure 2.3 for the result):

```
\begin{figure}[!ht]
```

```
  \fcapside{
    \caption[A figure with a side caption]{%
      This is a figure with a side caption.

      The caption is placed on the right side of the figure,
        and it can be as long as you want.

      You can also include multiple paragraphs in the caption,
        and it will be formatted nicely.
    }
    \label{fig:figure-with-side-caption}
  }{
    \includegraphics[width=0.4\textwidth]{example-image-golden}
  }
\end{figure}
```

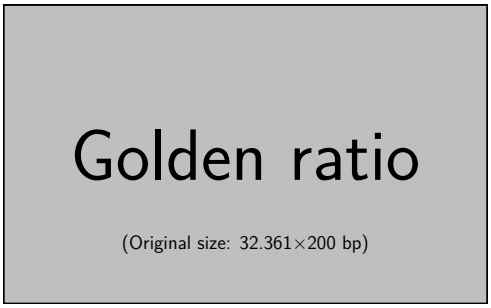

Figure 2.3 / This is a figure with a side caption.

The caption is placed on the right side of the figure, and it can be as long as you want.

You can also include multiple paragraphs in the caption, and it will be formatted nicely.

2.3 Powerful math macro helpers

The template provides many powerful macros to help you write math more conveniently and beautifully.

Math macros declaration

The `\Declare...` macros allow you to declare new math operators, categories, texts, and mathcal symbols with a simple syntax. For example, if you want to define `\CC` to be `\mathcal{C}`, you can simply write `\DeclareCals{C}`. See Table 2.4 for a summary of the available macros and their usage.

Custom quotient macro

The `\quotient` macro allows you to render pretty quotient expressions with customizable spacing and alignment. For example, `\quotient{X}{\sim}` renders as $X/_{\sim}$.

The full syntax of the `\quotient` macro is as follows:

```
  \quotient
    [<raise amount for numerator>] % default: 0.2
    [<negative space amount before slash>] % default: 1.7
    {<numerator>}
    {<denominator>}
```

Helper macro	Usage	Defined macro	Definition
`\DeclareOps`	`\DeclareOps{Hom}`	`\Hom`	`\operatorname{Hom}`
`\DeclareLimits`	`\DeclareLimits{colim}`	`\colim`	`\operatorname*{colim}`
`\DeclareCats`	`\DeclareCats{Set}`	`\Set`	`\mathbf{Set}`
`\DeclareTexts`	`\DeclareTexts{Spec}`	`\Spec`	`\mathrm{Spec}`
`\DeclareBbs`	`\DeclareBbs{R}`	`\R`	`\mathbb{R}`
`\DeclareCals`	`\DeclareCals{C}`	`\CC`	`\mathcal{C}`
`\DeclareFraks`	`\DeclareFraks{g}`	`\gg`	`\mathfrak{g}`

Table 2.4 / Math macros declaration helpers.

Stacked arrows

The `\doublestack`, `\triplestack`, and `\quadrustack` macros allow you to stack multiple symbols on top of each other with customizable spacing. When the stacked symbols are `\leftarrow` or `\rightarrow`, there are shortcuts `\doubleleftarrows`, `\doublerightarrows`, etc. Here is the effect:

$$A \to B \rightrightarrows C \overset{\rightarrow}{\underset{\rightarrow}{\rightarrow}} D \overset{\rightarrow}{\underset{\rightarrow}{\rightarrow}} E$$

2.4 Environments and cross-references

The template provides out-of-box environments for theorems, definitions, examples, remarks, etc., with a clean and elegant design. Use `\begin{theorem} ... \end{theorem}` to create a theorem environment, and similarly for other environments. Each environment has a starred version that is unnumbered, e.g., `\begin{theorem*} ... \end{theorem*}`.

You can add an optional argument (e.g. `\begin{theorem}[Fermat's Last Theorem]`) to specify a custom title for the environment, which will be displayed after the environment name and number.

To refer to these environments, first label them with `\label{...}` after the `\begin{...}` command, and then use `\cref{...}` or `\Cref{...}` to refer to them. The `\cref` command produces a lowercase reference, while `\Cref` produces a capitalized reference. These macros can also handle multiple references at once, e.g. `\cref{thm1,thm2}` will produce a reference to both `thm1` and `thm2`.

See Figure 2.5 for an example of using these environments and cross-references.

2.5 Beautiful layout and typography

The template provides out-of-box layout and typography settings that are designed to be clean, elegant, and easy to read. Here are most of the features:

- A4 paper size with reasonable margins.
- Reasonable line spacing and paragraph spacing for better readability.
- Fancy headers and footers that display the section name and page numbers.
- Beautiful title styles for sections and subsections.
- A clean and elegant design for the table of contents.

```
\begin{theorem} \label{thm:example-theorem}
  This is a numbered theorem.
\end{theorem}

\begin{theorem*}
  This is an unnumbered theorem.
\end{theorem*}

\begin{theorem}[Fermat's Last Theorem] \label{thm:fermats-last-theorem}
  There are no positive integers $a$, $b$, and $c$ such that $a^n + b^n = c^n$
    for any integer $n > 2$.
\end{theorem}

You can refer to the theorems above using \verb|\cref|:
    \cref{thm:example-theorem} and \verb|\Cerf|: \Cref{thm:example-theorem}.
You can also refer to several theorems at once:
    \cref{thm:example-theorem,thm:fermats-last-theorem}.
```

Theorem 2.1. *This is a numbered theorem.*

Theorem. *This is an unnumbered theorem.*

Theorem 2.2 (Fermat's Last Theorem)**.** *There are no positive integers a, b, and c such that $a^n + b^n = c^n$ for any integer $n > 2$.*

You can refer to the theorems above using `\cref`: theorem 2.1 and `\Cerf`: Theorem 2.1. You can also refer to several theorems at once: theorems 2.1 and 2.2.

Figure 2.5 / Example of theorem environments and cross-references.

- Special design for bibliography section, with index in typewriter font and data like arXiv, DOI, etc. displayed in a new line for better readability.

2.6 Git integration

The template is designed to work well with `git` for version control. The `.gitignore` file is set up to ignore auxiliary files generated by LaTeX, so that your repository stays clean and only contains the source files.

`pre-commit` is used to register hooks that run before each commit. See https://pre-commit.com/ for more details about `pre-commit`. The hooks are defined in the `.pre-commit-config.yaml` file, and they include:

- Check whether the exported `.png` files in `figures/inkscape` are up-to-date with the source `.svg` files, and if not, run the export script to update them.
- Check whether the output `.pdf` file is up-to-date with the source files.

By using these hooks, you can ensure that your exported figures and output PDF are always up-to-date with your source files, which can help prevent issues when pushing to a remote repository or sharing your work with others.

A GitHub action is also set up to automatically publish the PDF output to GitHub Pages whenever you push to the `main` branch, so that you can easily share your article with others by providing a link to the

published PDF. Since Inkscape is not available in the GitHub action environment, and the compilation of the document is not stable, all compilation steps should be done locally before pushing to the remote repository, and the intermediate files including exported `.png` files and the output `.pdf` file should be committed to the repository to ensure that the published PDF is always up-to-date with the source files. They are not ignored by `.gitignore` for this reason.

3 Usage

It is **strongly recommended** to use `sync-template`, which is a CLI tool that can automatically synchronize the template updates. Visit https://github.com/XXCC-Unicorn-Press/sync-template for more details.

3.1 Initialize a new project

First you need to initialize a new project with the template. You can do this by running the following command in your terminal:

```
sync-template init gh:XXCC-Unicorn-Press/article-template <project-directory>
```

Alternatively, you can also clone the template repository your self by running the following command:

```
git clone --depth 1 https://github.com/XXCC-Unicorn-Press/article-template.git
  <project-directory>
```

3.2 Build the project

After initializing the project, you can build it by running `latexmk` in the project directory. You will get a PDF file named `main.pdf` in the project directory after the build is complete. The default result is exactly this document you are reading.

3.3 Modify the metadata

You can modify the metadata in `preamble/meta.tex`. See Table 3.1 for the description of each metadata.

Metadata	Description
`\doctitle`	The title displayed on the title page. You can use text formats or math equations.
`\doctitlestring`	The title written in the PDF metadata. This should be a pure string.
`\docauthor`	The author. This should be a pure string

Table 3.1 / Metadata in `preamble/meta.tex`.

3.4 Write the content

The content of the paper is organized in the `content` directory. It is a good habit to create a new file per section and name the file after the section name.

You should `\input` the content files in `main.tex` in the order they appear in the paper.

3.5 Insert a figure

To insert a figure, first place the figure file in the `figures` directory, and then use the `\includegraphics` command to include the figure in the content file.

If you are using Inkscape to create your figures, you can place the source files in `figures/inkscape` and run `./figures/inkscape/export.sh` to export all the `.svg` files to `.png` files, which can be included in the document directly.

See Section 2.2 for more details on figure management.

3.6 Synchronize template updates

Sometimes the template may be updated, but your local copy will not be automatically synchronized.

If you are using `sync-template`, you can simply run `sync-template sync` in the project directory to synchronize the updates. Refer to https://github.com/XXCC-Unicorn-Press/sync-template for more details.

If you are not using `sync-template`, you can manually examine the updates in the template repository and apply the necessary changes to your local copy.

4 Some tips

This section provides some tips for article writing. It is **strongly recommended** to read this section before writing your article.

4.1 Filename convention

In LaTeX project, it is a good practice to name files according to the following convention:

- Use only lowercase letters, numbers, and hyphens (no spaces, underscores, or special characters).
- Make the filename descriptive and concise, reflecting the content of the file.
- Name after the section name if the file contains a specific section.

Here are some examples of good filenames:

- `introduction.tex`
- `some-tips.tex`
- `fermats-last-theorem.tex`

- `portrait-of-shakespeare.png`

Here are some examples of bad filenames:

- `section1.tex` (not descriptive, and the index may change)
- `some tips.tex` (contains spaces)
- `fermat's-last-theorem.tex` (contains apostrophe)
- `portrait-of-Shakespeare.png` (contains uppercase letters)

4.2 Cross-references

When you need to refer to a section, figure, theorem, or any other numbered element in your article, it is best to use **cross-references** instead of hardcoding the numbers. This way, if you add or remove sections or figures, the references will automatically update.

To create a cross-reference, you need to *label* the element you want to refer to, and then *refer* to it.

To label an element, use `\label{...}` command immediately after the element you want to label. If the element is a theorem (or other similar environment), you should place the `\label{...}` command right after the `\begin{...}` command. If the element is a figure, you should place the `\label{...}` command right after the `\caption{...}` command.

To refer to a labeled element, use the `\Cref{...}` command, which will automatically insert the correct number and the appropriate name (e.g., "Section", "Figure", "Theorem") based on the type of the labeled element.

It is a good practice to use a consistent prefix for labels to indicate the type of element being labeled. For example:

- `sec:` for sections (e.g., `sec:introduction`)
- `fig:` for figures (e.g., `fig:example`)
- `thm:` for theorems (e.g., `thm:example`)

Similar to filenames, the labels should be descriptive and concise, reflecting the content of the element being labeled, and they should be named in kebab-case.

Figure 4.1 is an example of how to use cross-references in LaTeX.

4.3 Citations

When you need to cite a reference in your article, it is best to use **citation keys** instead of hardcoding the reference information. This way, if you need to change the reference information (e.g., author name, title, year), you can simply update the bibliography file, and all citations will automatically update.

Suppose you have a bibliography item in your `references.bib` file like this:

```
\section{Introduction}
\label{sec:introduction} % section name is often long, so we put \label on the next line

\begin{figure}
  \includegraphics{example-image-golden}
  \caption{An example figure.}
  \label{fig:example} % caption is often long, so we put \label on the next line
\end{figure}

\begin{theorem} \label{thm:example} % we put \label right after \begin{theorem}
  This is an example theorem.
\end{theorem}

In \Cref{sec:introduction}, we display an example figure in \Cref{fig:example} and state
    an example theorem in \Cref{thm:example}.
```

Figure 4.1 / An example of using cross-references.

```
@book{example-book,
  author = {A Person},
  title = {A Book},
  % ... other info ...
}
```

You can cite this reference in your article using `\cite{example-book}` command. This will automatically insert the correct citation index wrapped by brackets (e.g., `[1]`). You can also specify some additional data by typing `\cite[...]{example-book}`, and the additional data will be included in the brackets (e.g., `[1, Section 2.1]`).

4.4 Figures

Figures can be inserted by using `\begin{figure} ... \end{figure}` environment. In this environment, there are usually three lines:

- `\includegraphics`, which specifies the image to insert and its size.
- `\caption`, which specifies the caption of the figure.
- `\label`, which specifies the label for cross-referencing the figure.

Remember to place the `\label` command right after the `\caption` command, so that the cross-reference will display the correct figure number.

If your caption has multiple paragraphs (contain a blank line), you have to specify a short caption by using `\caption[Short caption]{Long caption}`. Otherwise, it will cause an error.

You may also want to use side captions for your figures. See Section 2.2 for more details.

References

[1] XXCC Unicorn Press. *Article Template*. 2026.
URL: https://github.com/XXCC-Unicorn-Press/article-template.

[2] XXCC Unicorn Press. *Journal Template*. 2026.
URL: https://github.com/XXCC-Unicorn-Press/journal-template.

[3] jdujava. *TeXtured Template*. 2025.
URL: https://github.com/jdujava/TeXtured.